

202044502
Programming with
JAVA

Unit -2
(Part -2)
Topic – Interfaces

Academic Year: 2024-25 (II)

Prepared By:
Prof. Pratik B. Soni
(Asst. Prof. – I.T. Dept, MBIT)

1

Interfaces

- Interfaces Basics
- interface naming conflicts
 - method naming conflict
 - variable naming conflicts
- marker interface
- interface vs abstract class vs concrete class
- Coupling and typecasting

2

Interfaces

- **Interfaces Basics**
- interface naming conflicts
 - method naming conflict
 - variable naming conflicts
- marker interface
- interface vs abstract class vs concrete class
- Coupling and typecasting

3

Interfaces - Basics

- Interfaces are designed to support dynamic method resolution at run time.

Syntax:

```
<access> interface <name>
{
    type final-varname1 = value;
    type final-varname2 = value;
    // ....
    type final-varnameN = value;

    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    //....
    return-type method-nameN(parameter-list);
}
```

Important Points:

- "interface" keyword instead of "class"
- All Methods are abstracted in Interface, But not prefixed by abstract keyword.
- Variables are implicitly "final" and "static".
- Interfaces can be use by other classes using "implements" keyword.
- It cannot be instantiated just like the abstract class.
- It can be implemented through implemented references.
- When you implement an interface method, it must be declared as **public**.

4

Interfaces - Basics

```
interface Drawable
```

```
{
    void draw();
}
```

```
class Rectangle implements Drawable
```

```
{
    public void draw()
    {
        System.out.println("drawing rectangle");
    }
}
```

```
class Circle implements Drawable
```

```
{
    public void draw()
    {
        System.out.println("drawing circle");
    }
}
```

```
class Test1
```

```
{
    public static void main(String args[])
    {
        Drawable d=new Rectangle();
        d.draw();

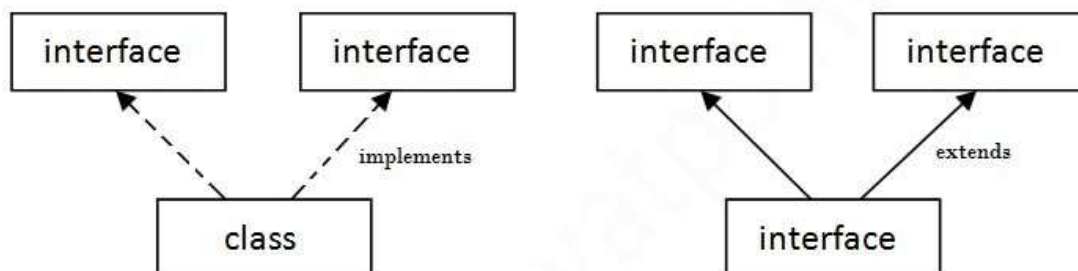
        Circle c=new Circle();

        d=c;
        d.draw();
    }
}
```

```
drawing rectangle
drawing circle
```

5

Interfaces – Basics – Multiple Inheritance



6

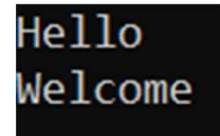
Interfaces – Basics – Multiple Inheritance

```
interface A
{
    void print();
}
```

```
interface B
{
    void show();
}
```

```
class C implements A,B
{
    public void print()
    {
        System.out.println("Hello");
    }
    public void show()
    {
        System.out.println("Welcome");
    }
}
```

```
class Test2
{
    public static void main(String args[])
    {
        C obj = new C();
        obj.print();
        obj.show();
    }
}
```



Hello
Welcome

7

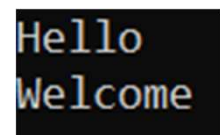
Interfaces – Basics – Multiple Inheritance

```
interface A
{
    void print();
}
```

```
interface B extends A
{
    void show();
}
```

```
class C implements B
{
    public void print()
    {
        System.out.println("Hello");
    }
    public void show()
    {
        System.out.println("Welcome");
    }
}
```

```
class Test3
{
    public static void main(String args[])
    {
        C obj = new C();
        obj.print();
        obj.show();
    }
}
```



Hello
Welcome

8

Interfaces

- Interfaces Basics
- **interface naming conflicts**
 - **method naming conflict**
 - **variable naming conflicts**
- marker interface
- interface vs abstract class vs concrete class
- Coupling and typecasting

9

Interfaces – method naming conflicts

- When Both Interfaces **have same method name and same definition** – it's called naming conflicts. (like overriding) – **ONLY ONE TO BE OVERRIDED.**
- When Both Interfaces have **same method name and different definition** – it's called naming conflicts. (like overloading) – **ALL DEFINITIONS ARE OVERRIDED.**

```

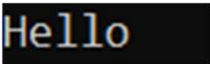
interface A
{
    void print();
}

interface B
{
    void print();
}

class C implements A,B
{
    public void print()
    {
        System.out.println("Hello");
    }
}
  
```

```

class Test4
{
    public static void main(String args[])
    {
        C obj = new C();
        obj.print();
    }
}
  
```



Hello

10

Interfaces – variable naming conflicts

```
interface data1
{
    String m1 = "first";
}

interface data2
{
    String m1 = "second";
}

public class Main1 implements data1, data2 {
    public static void main(String []args)
    {
        System.out.println(data1.m1);
        System.out.println(data2.m1);
    }
}
```

OUTPUT:

```
first
second
```

11

Interfaces

- Interfaces Basics
- interface naming conflicts
 - method naming conflict
 - variable naming conflicts
- **marker interface – LATER ON**
- interface vs abstract class vs concrete class
- Coupling and typecasting

12

Interfaces

- Interfaces Basics
- interface naming conflicts
 - method naming conflict
 - variable naming conflicts
- marker interface
- **interface vs abstract class vs concrete class**
- Coupling and typecasting

13

Interfaces – interface vs abstract class vs concrete class

- **Concrete Class:**
 - A concrete class in java is a class that has all its methods implemented. For any class to be classified as concrete, it cannot have any unimplemented methods.
 - All properties , Extending , implementing are there with all basic requirements.

14

Interfaces – interface vs abstract class vs concrete class

Interfaces	Abstract Class	Concrete Class
Interface can have all abstracted methods only.	Abstract class can have both an abstract as well as concrete methods.	A concrete class can only have concrete methods. Even a single abstract method makes the class abstract.
No instantiation is allowed.	Abstract class can not be instantiated using new keyword.	Concrete class can be instantiated using new keyword.
Interface does not declare as “final”.	Abstract class can not be declared as a final class.	Concrete class can be declared final.
It declares with “interface” keyword.	Abstract class declared using abstract keyword.	Concrete class is not having abstract keyword during declaration.
Interface can implement one / more other interfaces	Abstract class can inherit another class using extends keyword and implement an interface.	Interface can inherit only an interface.

15

Interfaces

- Interfaces Basics
- interface naming conflicts
 - method naming conflict
 - variable naming conflicts
- marker interface
- interface vs abstract class vs concrete class
- **Coupling and typecasting**

16

Interfaces – coupling

- Coupling, in software engineering, describes how tightly or loosely different modules of a system are connected.
- It reflects the dependency between classes, methods, or other components.
- **High coupling (Tight Coupling)** means that a change in one module will likely require changes in others, making the system harder to maintain and evolve.
- Conversely, **low coupling (Loose Coupling)** means that modules can operate independently, enhancing modularity and simplifying maintenance.

17

Interfaces – coupling

Tight Coupling

```
class Box
{
    public int vol;
    Box(int l, int w, int h)
    {
        this.vol = l * w * h;
    }
}

class Main1
{
    public static void main(String args[])
    {
        Box b = new Box(5,5,5);
        System.out.println(b.vol);
    }
}
```

Loose Coupling

```
interface vol {
    int volResult(int,int,int);
}

class Box implements vol {
    public int volResult(int l, int w, int h) {
        return l * w * h;
    }
}

public class Main1{
    public static void main(String args[]) {

        vol calVolume = new Box();
        int vol = calVolume.volResult(5, 5, 5);
        System.out.println(vol);
    }
}
```

18

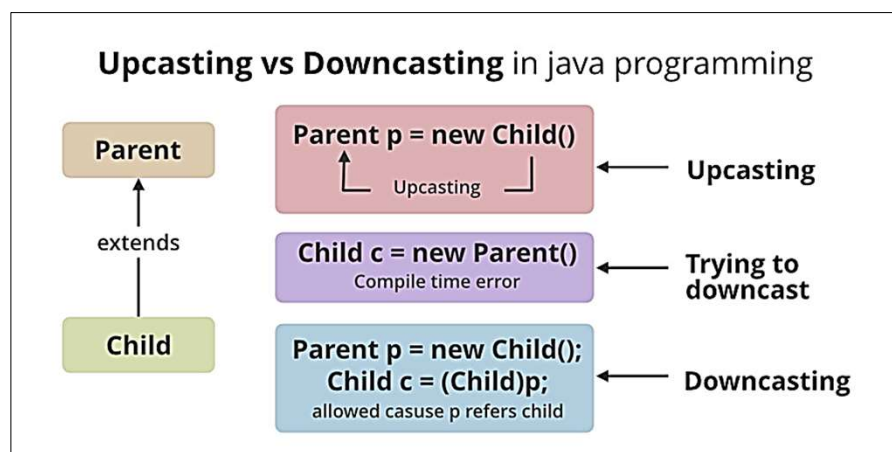
Interfaces

- Interfaces Basics
- interface naming conflicts
 - method naming conflict
 - variable naming conflicts
- marker interface
- interface vs abstract class vs concrete class
- **Coupling and typecasting**

19

Interfaces –type-casting

Up-Casting: Casting from a subclass to a superclass is called upcasting.



Down-Casting: It's the casting from a superclass to a subclass.

20

Interfaces – type-casting

```
public class Animal
{
    public void eat() {
        // ...
    }
}

public class Cat extends Animal
{
    public void eat() {
        // ...
    }
    public void meow() {
        // ...
    }
}

Cat cat = new Cat();

Animal animal = cat;
//OR//
Animal animal;
animal = (Animal) cat;
```

```
public class Animal
{
    public void eat() {
        // ...
    }
}

public class Cat extends Animal
{
    public void eat() {
        // ...
    }
    public void meow() {
        // ...
    }
}

Animal animal = new Cat();
((Cat) animal).meow();
```